

大家好，我是栾智健，今天我要分享的内容是，利用数学建模和 lua 仿真的方式方法，完成泊位问题。在物流信息管理课程的学习过程当中，我体会到，仿真往往具备更强的说服力了。因此，我以课上所学代码以及逻辑思维为底，完成本次的创新分享。

首先是对问题进行简单描述，一个典型的资源分配问题，也就是泊位分配：给定一组异构的待靠泊的船舶和一组有限的泊位，该如何决策？如 ppt 所示，这不仅是谁停在哪里的问题，还涉及到时间维度的排序。目标就是最小化总运营成本。挑战在于船舶大小不同、吃水不同，且泊位的装卸效率和转运成本各异。我们需要在等待时间、作业成本和泊位闲置成本之间寻找平衡。”

为了构建这个问题的数学模型，我定义了这些参数，总的来讲可以分为船舶参数和泊位参数。尤其是，特意考虑了船长和吃水与泊位长度和水深的匹配。这些参数构成了后续数学建模的基础。”

这是我们模型的核心，目标函数。也就是我 Lua 代码中 CalculateZ 函数的数学原型。我的目标是最小化总成本，它由三部分组成：

1. 作业与转运成本：即 $\sum \sum (c_{1jb} + c_{2jb})y_{jb}$ ，这部分反映了操作层面的开销。

2. 时间成本：即 $c_3 \times (\text{所有船舶的完工时间} + \text{转运时间})$ ，这里我引入了单位时间成本 c_3 来惩罚延误。

3. 固定成本： $\sum c_{4b}\alpha_b$ ，如果启用了泊位 b ，就要支付固定开销，这能避免滥用泊位资源。这个公式直接决定了算法优化的方向。”

有了目标之后就要有约束，我将其分为两大类：

1. 物理空间约束：这是硬指标。每艘船必须且只能分配一个泊位 ($\sum y_{jb} = 1$)；且船长不能超过泊位长 ($l_j \leq L_b$)，吃水不能超过水深 ($d_j \leq D_b$)。

2. 时间与逻辑约束：这是建模的难点。为了防止同一泊位上的船舶作业时间重叠，我使用了大M法。如公式(2)和(3)所示，通过引入0-1变量 x_{ij} ，确保如果两艘船在同一泊位，必须存在先后顺序，避免时间冲突。”

“模型建立后，如何求解？考虑到 NP-Hard 特性，我选择了回溯算法以寻找全局最优解。如图所示，算法从第一艘船开始尝试分配泊位，形成一棵决策树。为了提高效率，我在代码中加入了减枝的策略：在分配时立即检查物理约束（如船长是否超限），如果不满足，直接在这个节点‘剪掉’，不再向下搜索。这大大减少了计算量。”

“接下来进入代码实现环节。为了保证仿真的随机性和鲁棒性，我没有使用死数据。在 Lua 中，我使用 `math.randomseed(os.time())` 根据当前时间生成随机种子。`GenerateData` 函数会随机生成泊位的长度、水深，以及船舶的到达时间 和作业参数 。

大家可以看到，在这页我举了个例子，展示了将数学语言翻译成编程语言。可以看到函数中的这段逻辑：`checkLength`，`checkDraft`
这两行代码复现了数学模型中的公式(7)和公式(8) 。只有当 `checkLength` 和 `checkDraft` 都为 `true` 时，程序才会继续递归求解 。除了展示的部分，所有约束都被完美复现。

“这是算法运行后的控制台输出结果
上半部分打印了生成的参数，下半部分是**最终优化结果** 。
可以看到，系统会计算出最小目标函数值。
详细调度方案列出了每艘船 被分配到了哪个泊位，以及精确的开始 和结束时间 () 。这个调度表就是接下来可视化驱动的核心数据源。”

前面提到，只有仿真才有更强的说服力。首先，我根据老师课上所讲，在 MicroCity 中，建立了一个三维坐标系 。然后划分为两个区

- 泊位区：位于 $Z = 0$ 的岸线，我根据泊位数量计算坐标 `bx`，并加载了岸桥模型 `rmqc.glb`。
• 等待区：位于 $Z = 500$ 的远海区域，所有生成的船舶初始都停在这里等待。代码中，我通过 `shipViz` 表来管理所有船舶对象的状态（等待、靠泊、离港），为动画做好了准备。”

“这是最终的仿真效果。
当点击运行后，时间轴 `simTime` 开始流动 。Lua 脚本会实时遍历所有船舶，一旦当前时间进入某艘船的作业时间窗，该船就会从远海自动驶向指定泊位，并在作业完成后驶离
这不仅验证了调度方案的可行性，也直观展示了港口的动态作业过程。”

通过这次，在老师指导下的拓展学习，我不仅进一步掌握了泊位分配的数学建模方法，更熟练了如何用 Lua 语言在 MicroCity 中实现算法与可视化的结合。
以上就是我的分享，感谢大家，希望得到大家的指导。